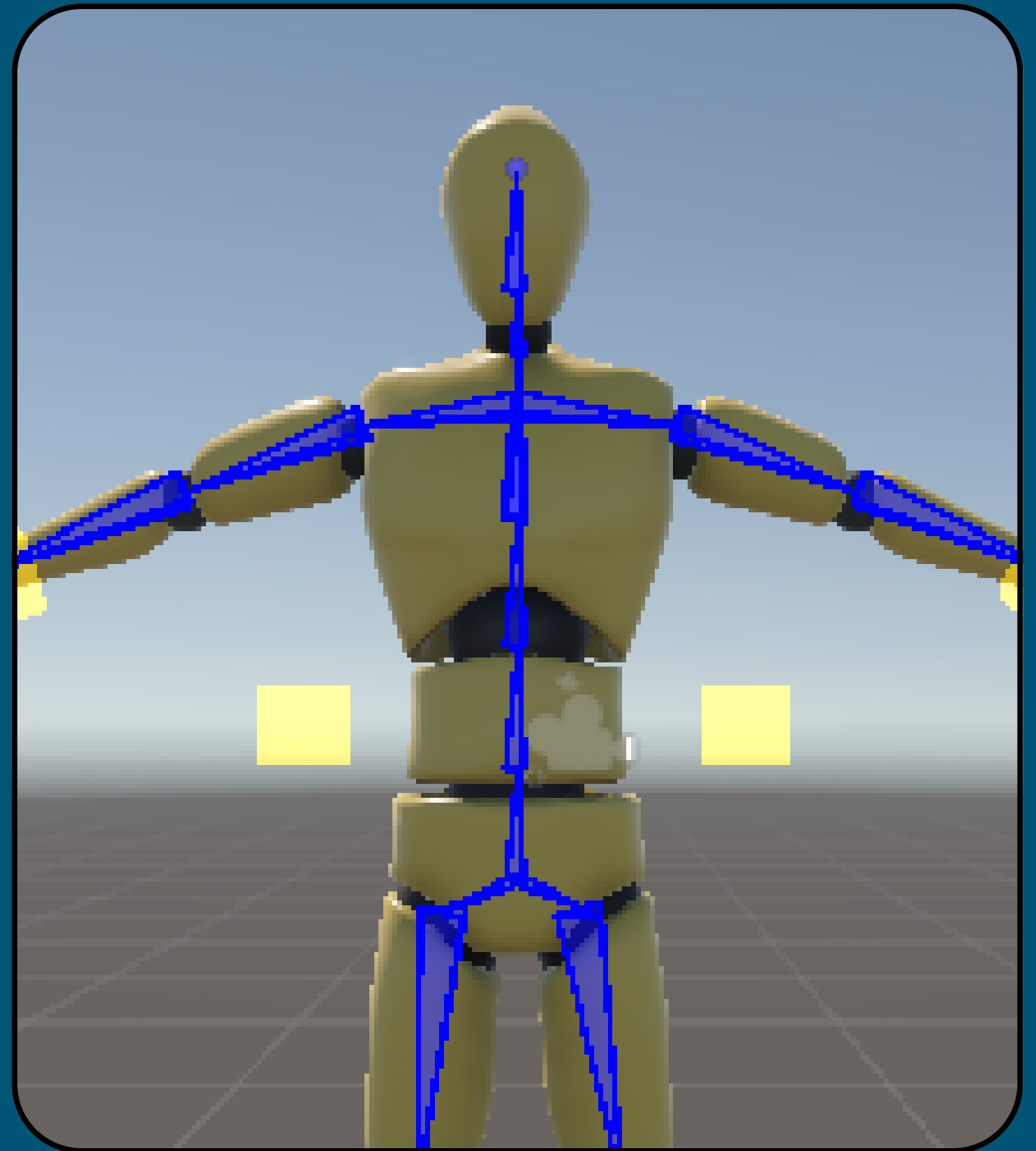
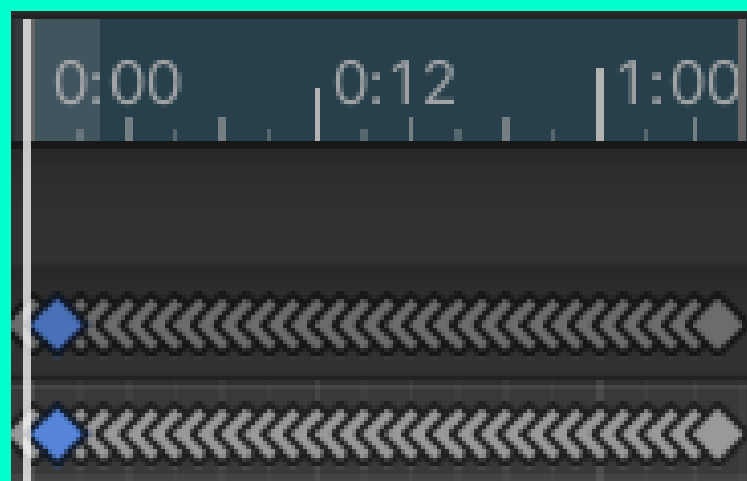


Animacão Procedural

Marco di Iorio



Keyframed Animations



Animação por Keyframes refere-se ao trabalho de manualmente criar posições intermediárias (keyframes) de um objeto ou esqueleto, para que algum programa faça a interpolação destes keyframes em uma animação [1].

Animação Procedural refere-se a qualquer animação dirigida por meio de código executado em tempo real ao invés de keyframes[2], onde as animações podem se adaptar baseado em parâmetros e condições de forma a tornarem-se mais realistas e adaptáveis ao ambiente[4].

Procedural Animation



Exemplo de Animação Procedural:



Uncharted 4

Personagem move a mão em direção à parede, deixa ela passar, e depois volta o braço à posição normal.

Forward Kinematics (FK) vs Inverse Kinematics (IK)

Cinemática Direta (FK):

Consiste em ajustar as rotações das articulações valores específicos. Como cada junta depende da rotação de seu parente, a posição final é resultado das manipulações dentre todas elas, de cima para baixo[3].

Cinemática Inversa (IK):

Utiliza um ponto definido no espaço, e calcula como as articulações devem ser rotacionadas para que apontem para este ponto. Isso é útil para fazer um personagem apontar a algum lugar ou agarrar algo[3].

O uso da Cinemática Inversa (IK) facilita a criação de animações procedurais por possibilitar torná-las mais adaptáveis ao ambiente em que o personagem está inserido. Portanto, será a técnica escolhida para o protótipo.

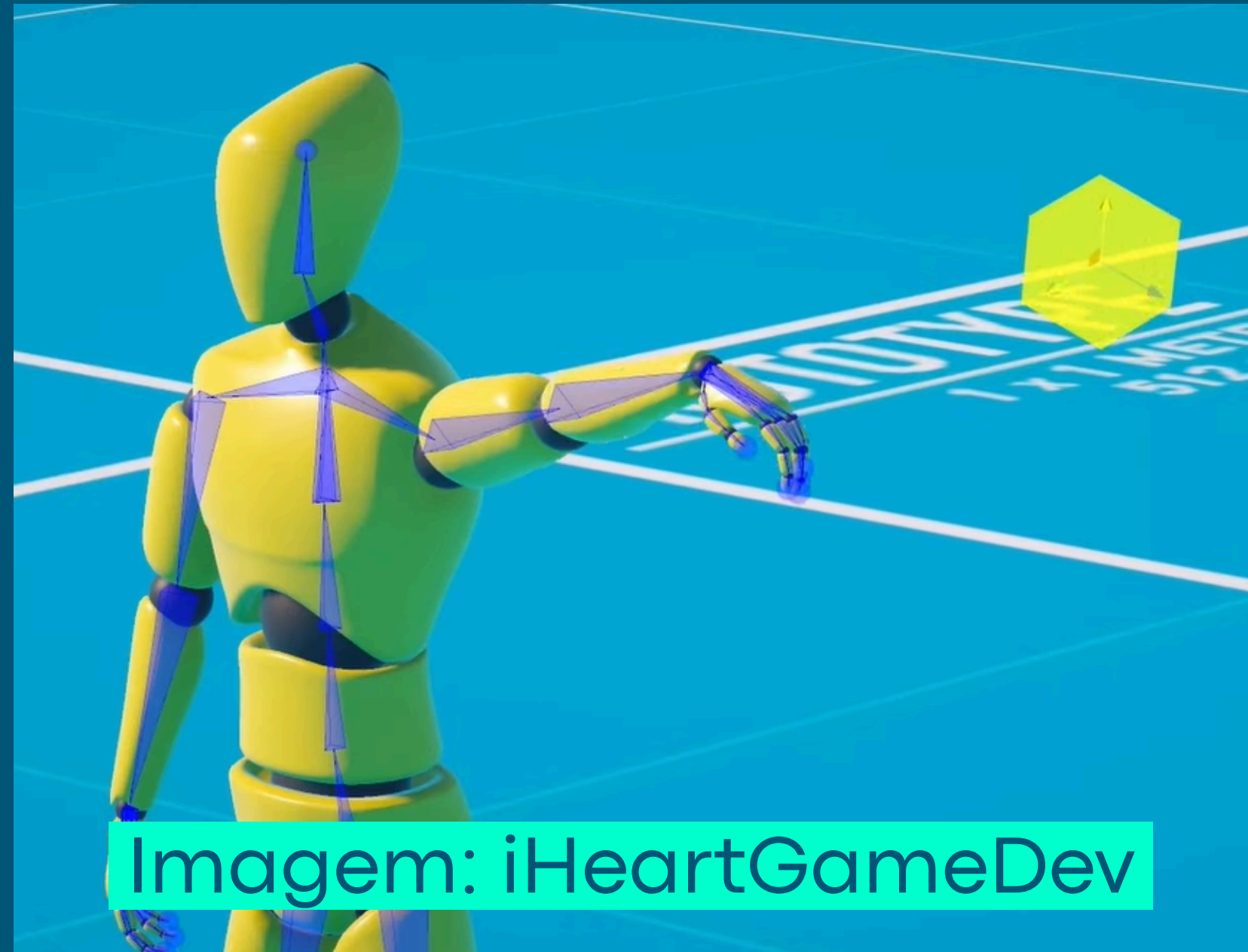
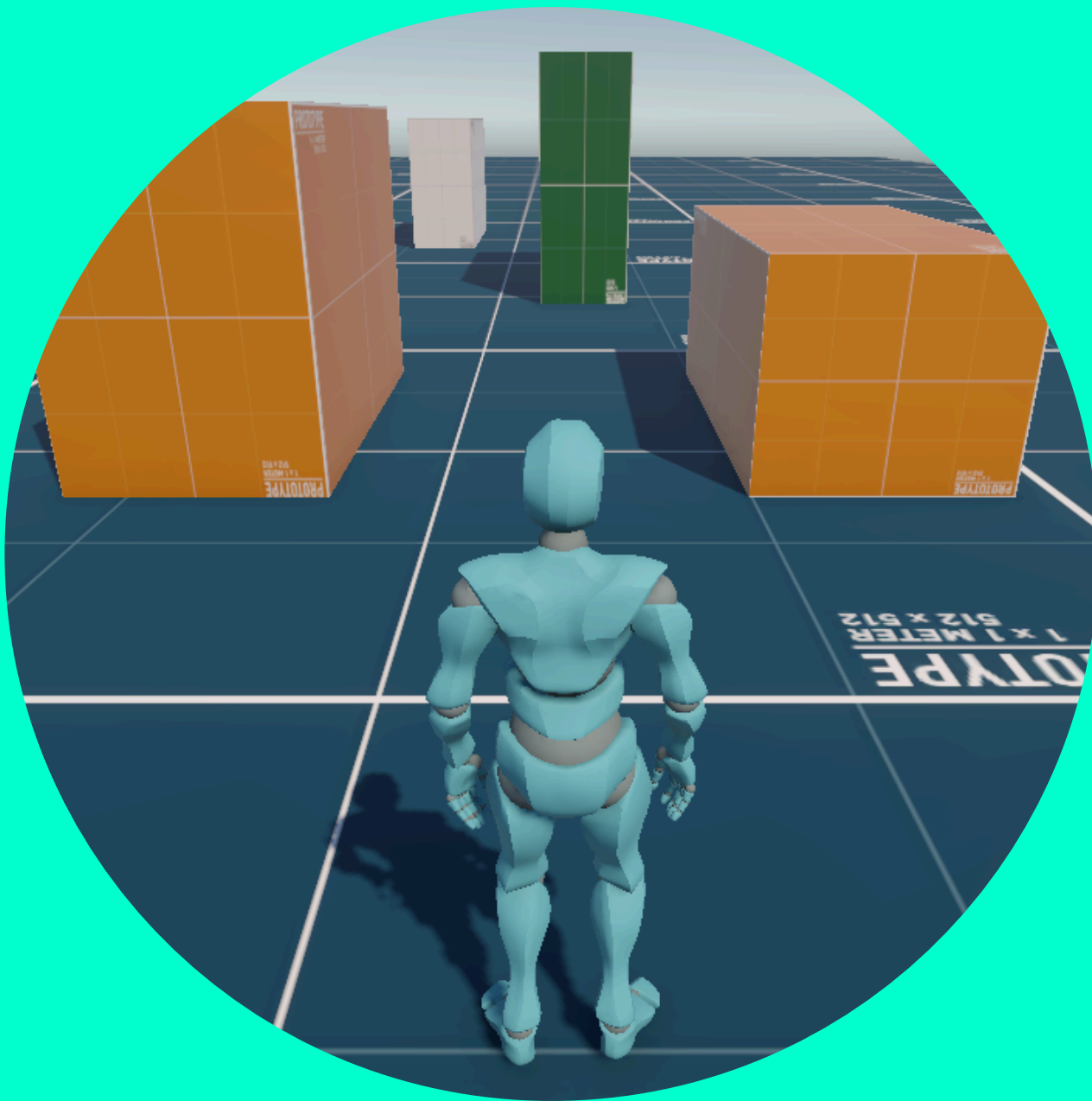


Imagem: iHeartGameDev

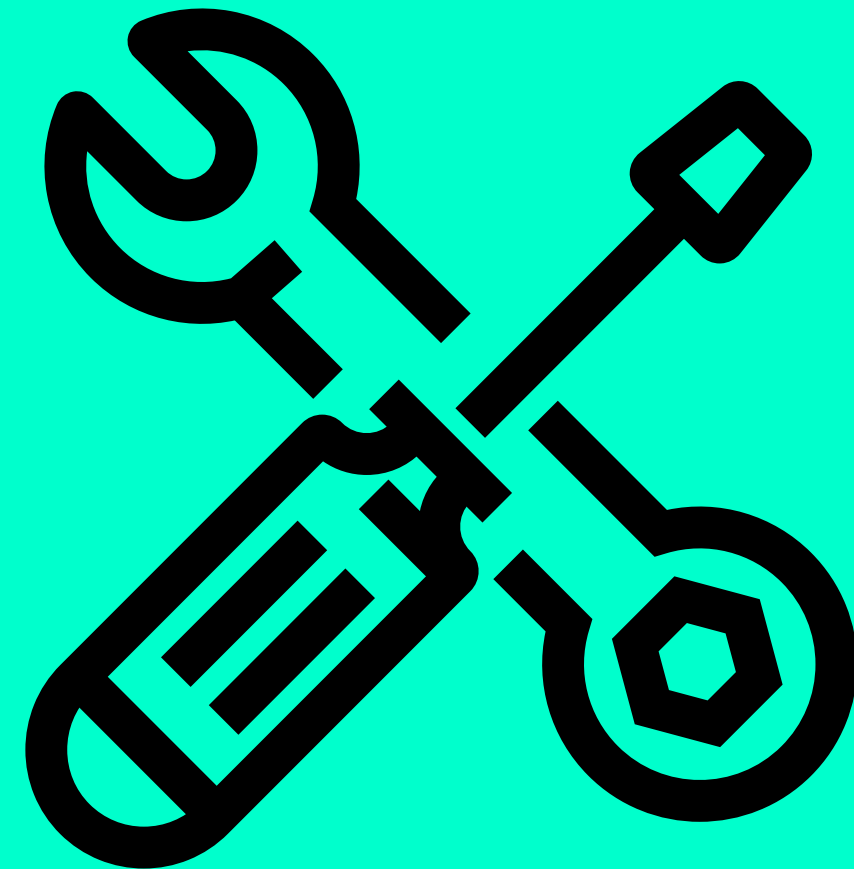


O Protótipo

Um pequeno playground com paredes posicionadas. Personagem jogável. Ao aproximar-se de qualquer parede, ele deve, proceduralmente, mover seu braço de forma a encostar na parede.

Ferramentas

- Unity Engine 6
- C#
- Visual Studio Code
- GitHub




```
using UnityEngine;  
using UnityEngine.Animations.Rigging;
```

Animation Rigging

1.3.1 · September 19, 2025

From Unity Registry by Unity Technologies Inc.

[com.unity.animation.rigging](#)

[Documentation](#)

[Changelog](#)

[Licenses](#)

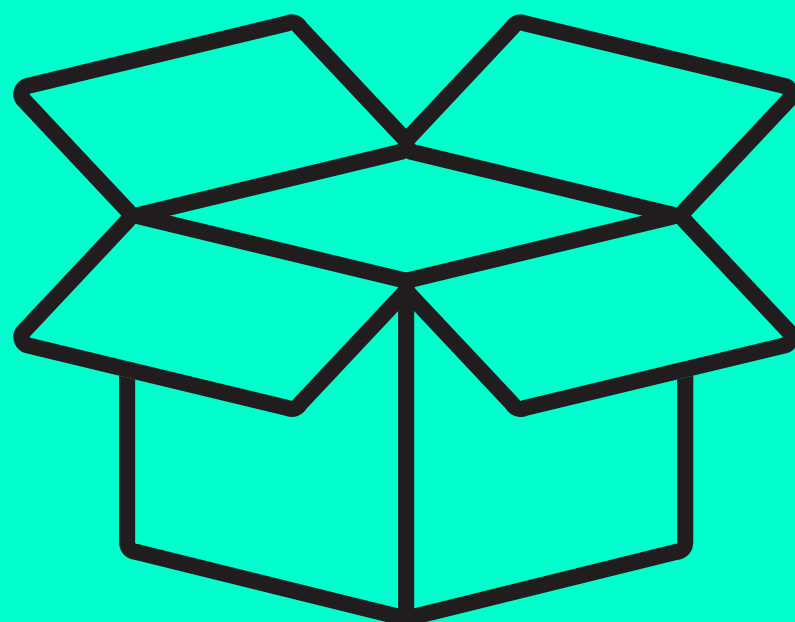
Install

Description

Version History

Dependencies

Animation Rigging toolkit using Unity's Animation C# Jobs



Ferramentas

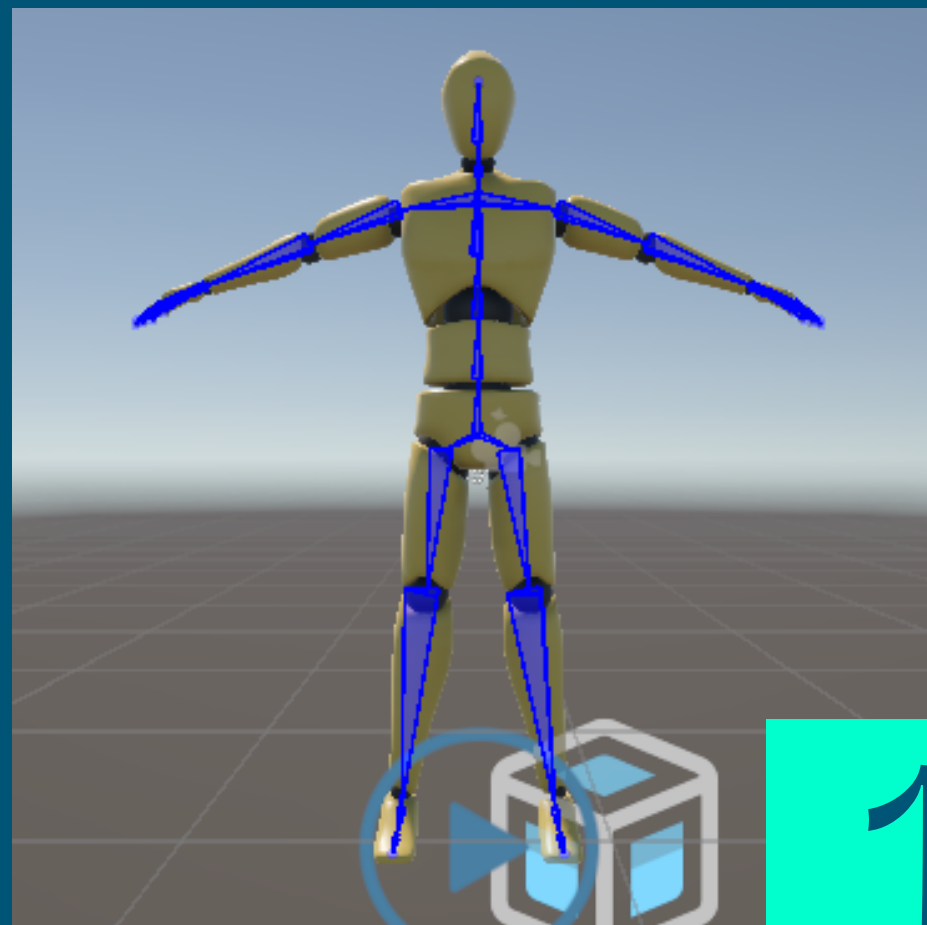
- **Animation Rigging:** Pacote built-in da Unity. Dá acesso a bibliotecas e componentes que tornam o processo de animação procedural muito mais fácil. Interage diretamente com o sistema de animação da Unity [4].

Desenvolvimento

Passo a Passo

Desenvolvimento

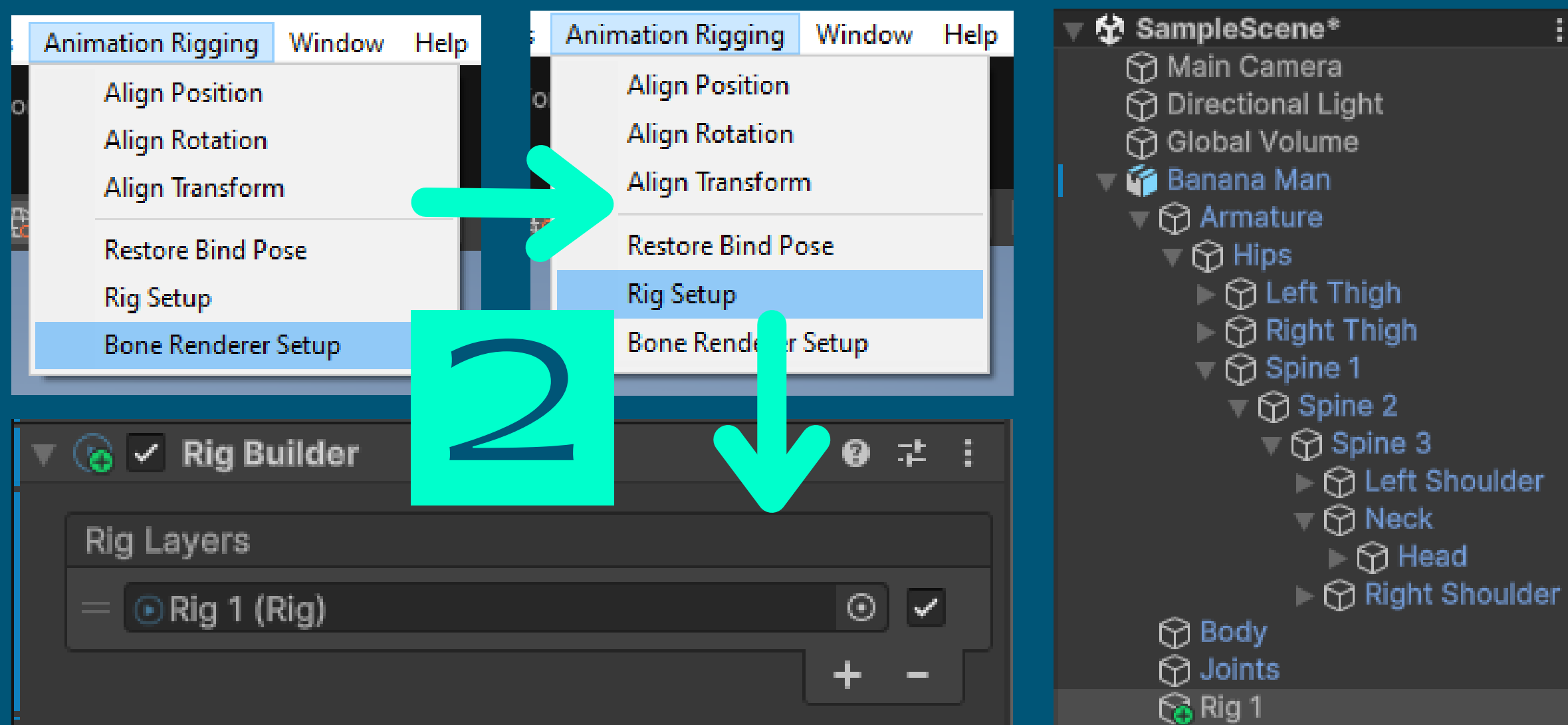
Colocar Rig em cena.



1

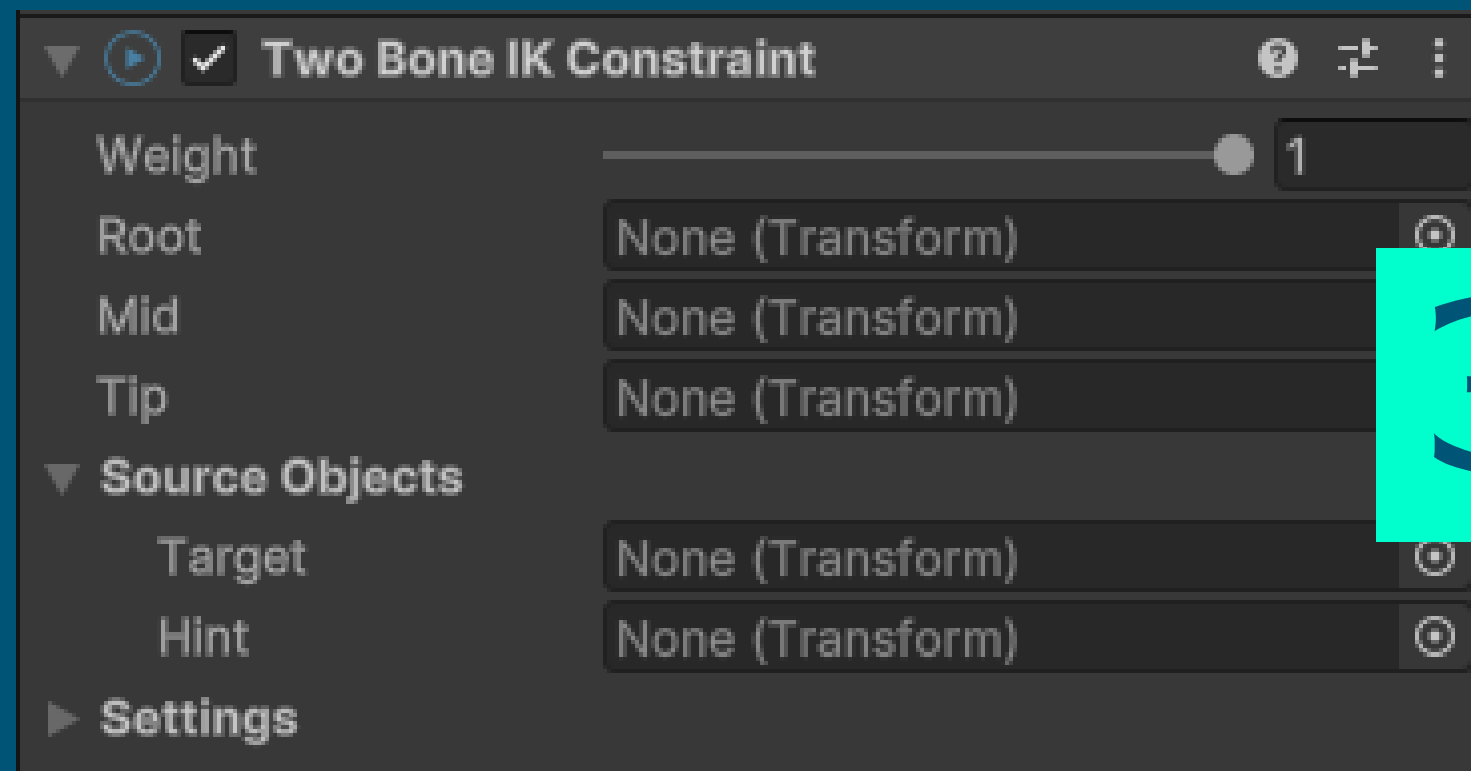
Desenvolvimento

Configurar Rig

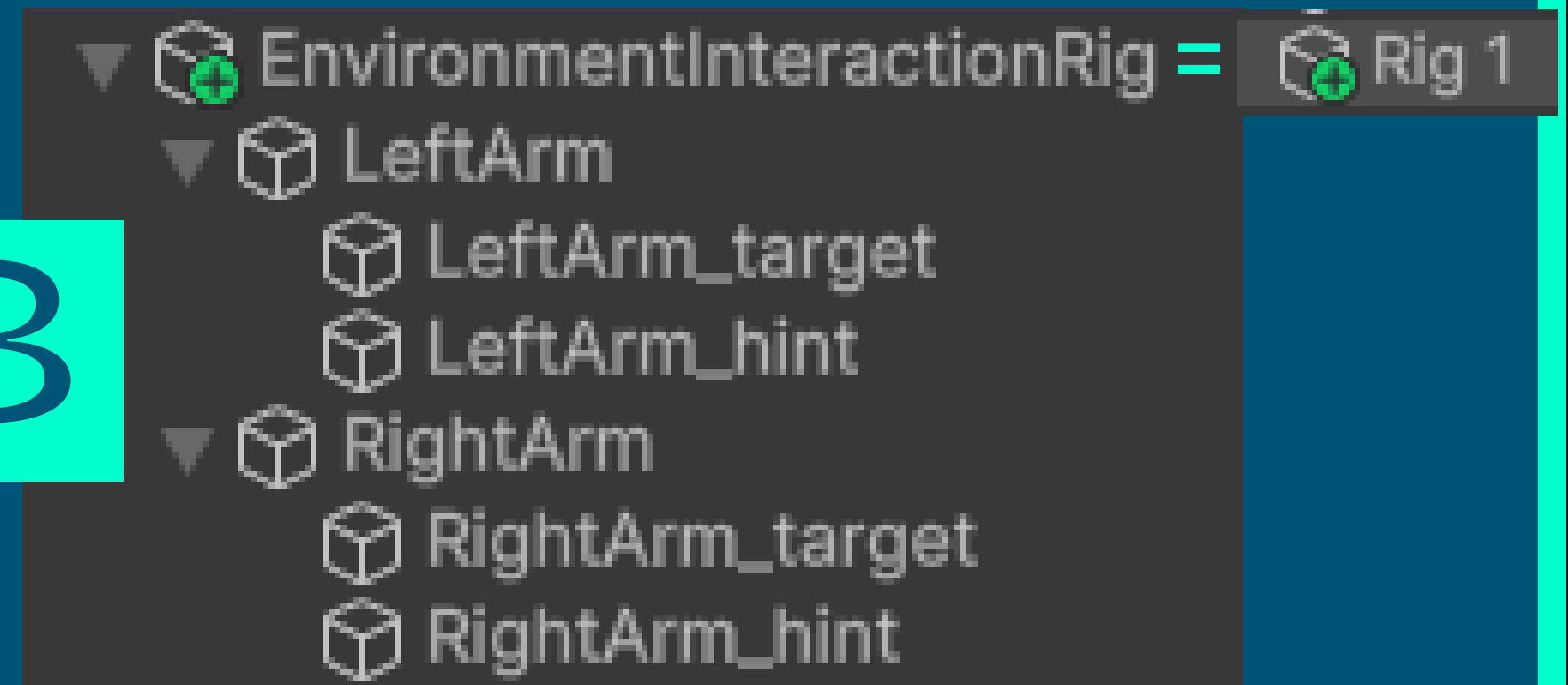


Desenvolvimento

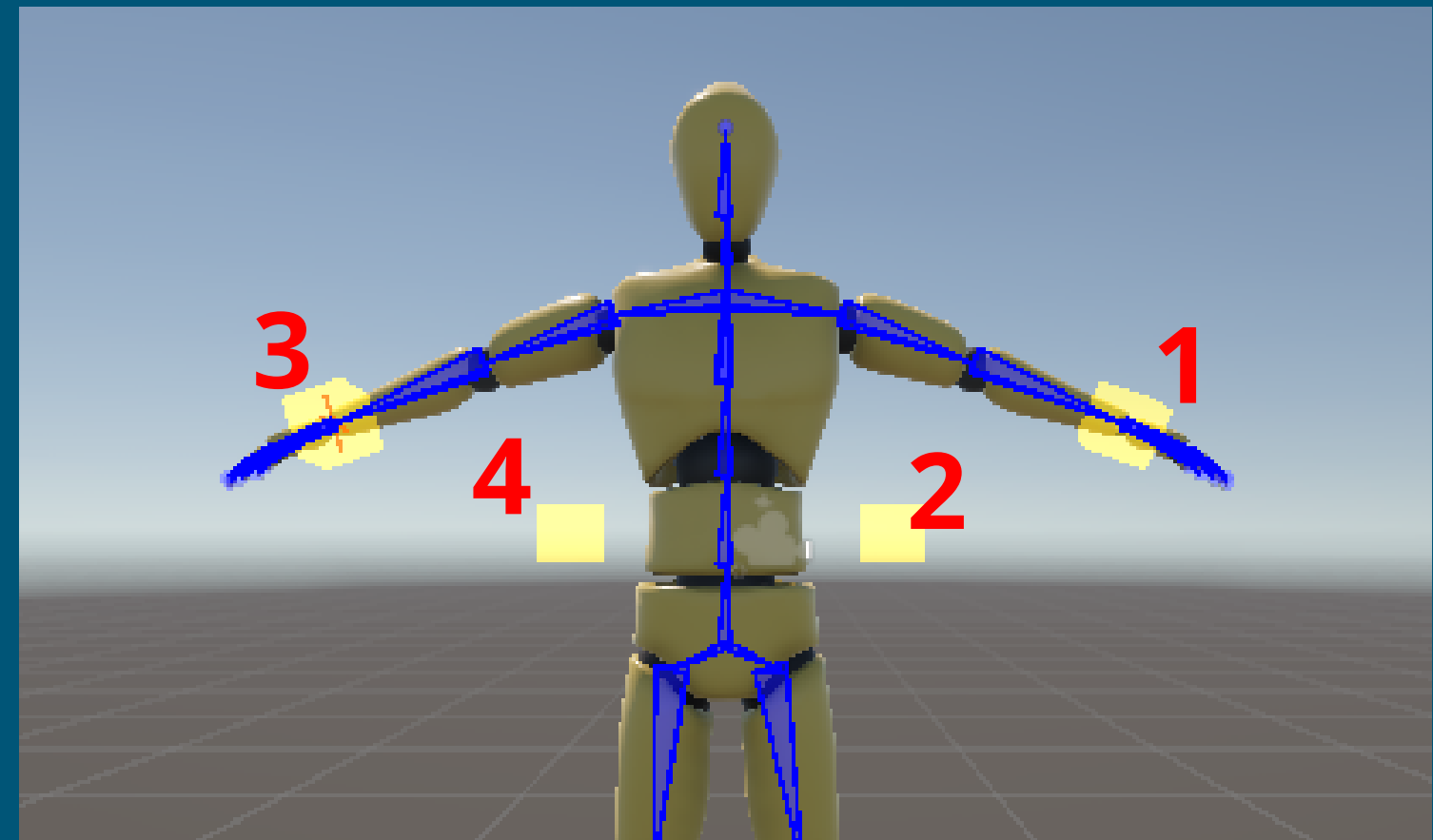
Adicionar um componente TwoBoneIKConstraint aos dois braços e configurá-los.



3

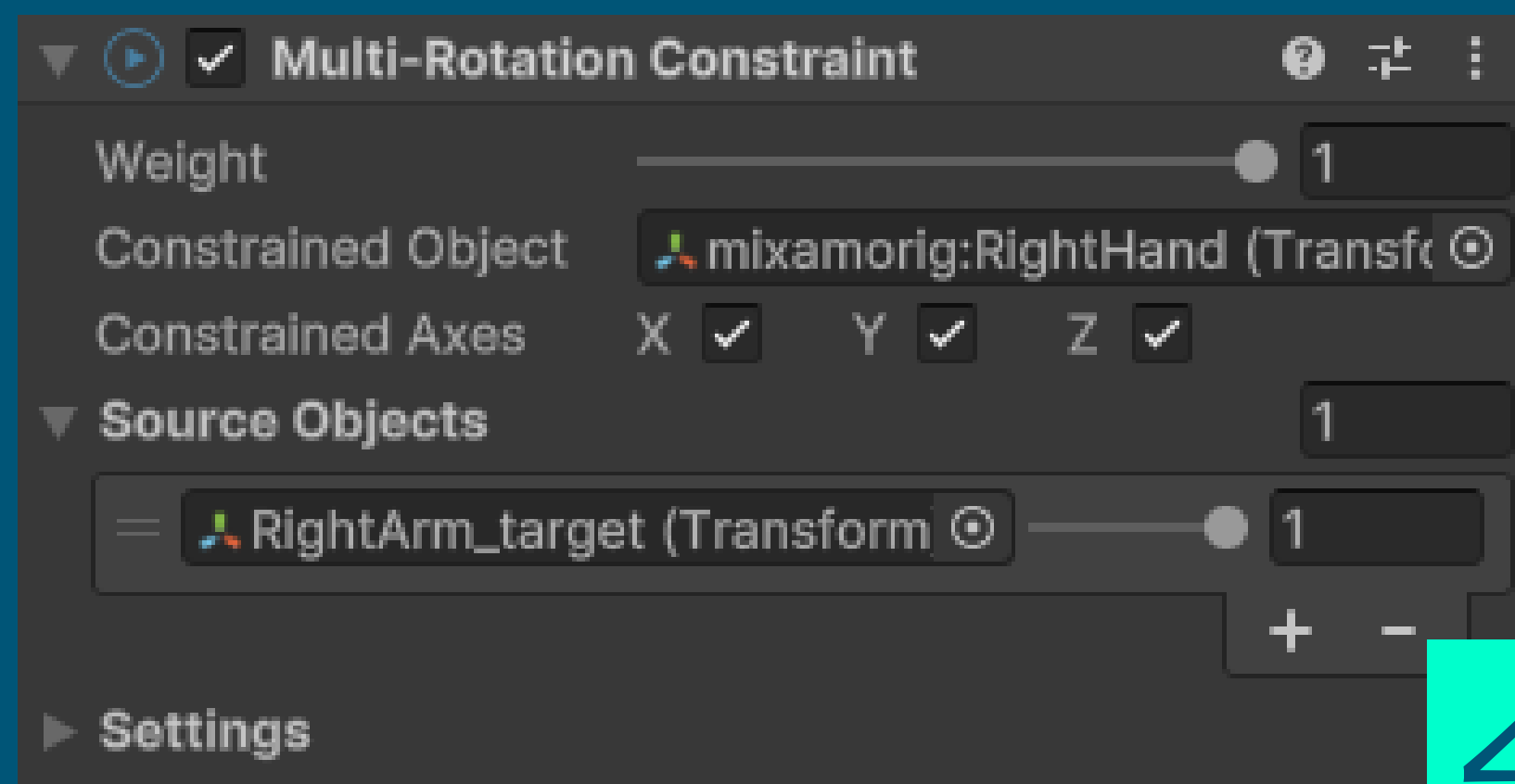


- ▼ EnvironmentInteractionRig
 - ▼ LeftArm
 - LeftArm_target 1
 - LeftArm_hint 2
 - ▼ RightArm
 - RightArm_target 3
 - RightArm_hint 4



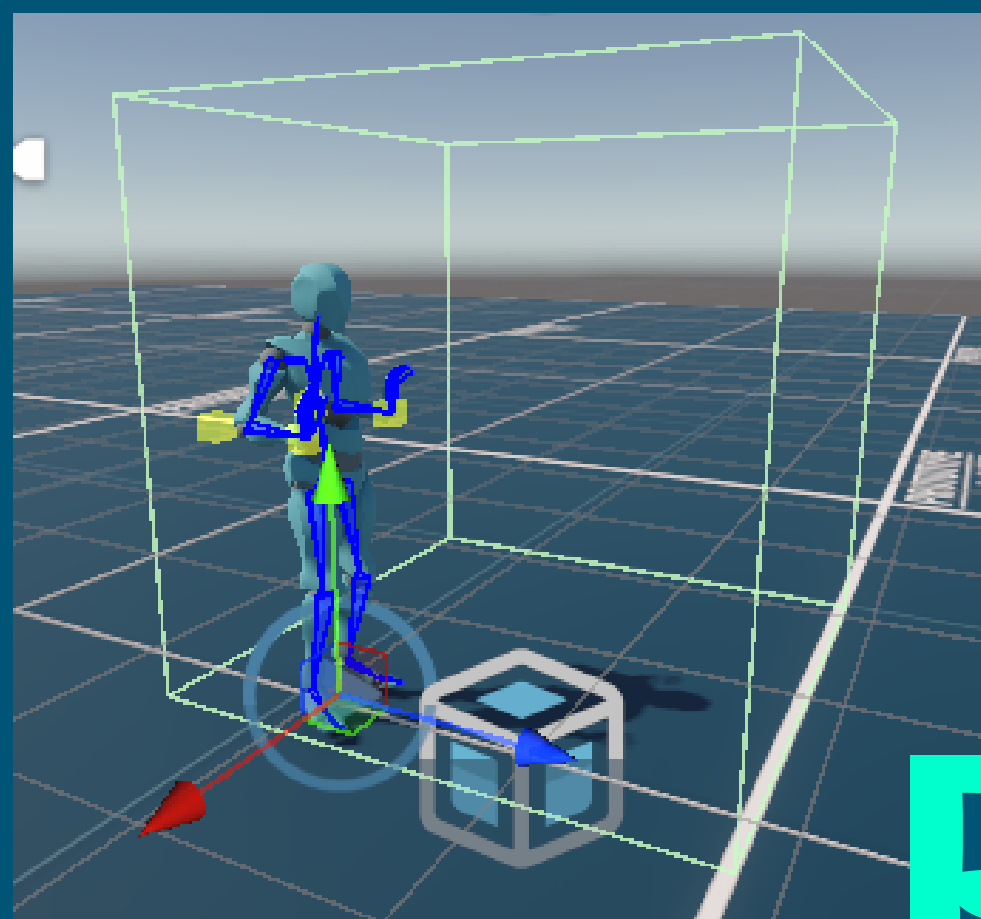
Desenvolvimento

Adicionar o componente MultiRotationConstraint para ambos os braços.



4

Desenvolvimento



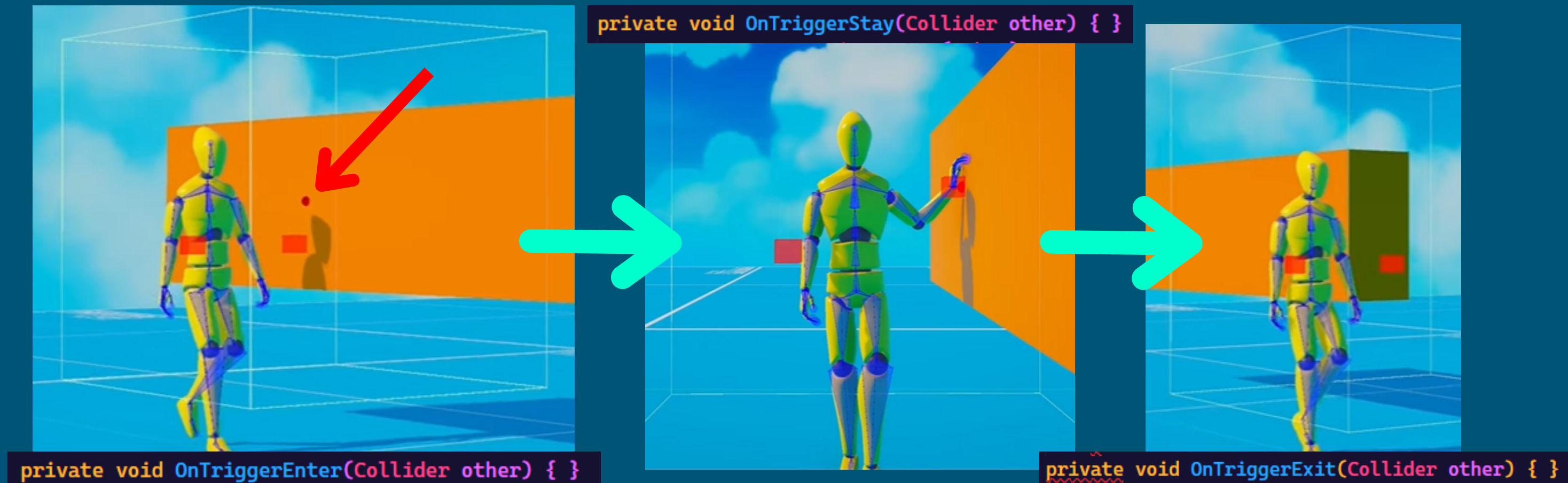
Criar uma área de detecção, para que superfícies que adentrem-na sejam detectadas.

```
1 reference
private void ConstructEnvironmentDetectionCollider() {
    float wingspan = _rootCollider.height;

    BoxCollider boxCollider = gameObject.AddComponent<BoxCollider>();
    boxCollider.size = new(wingspan, wingspan, wingspan);
    boxCollider.center = new(_rootCollider.center.x, _rootCollider.center.y + (wingspan * .25f),
        _rootCollider.center.z + (wingspan * .5f));
    boxCollider.isTrigger = true;
}
```

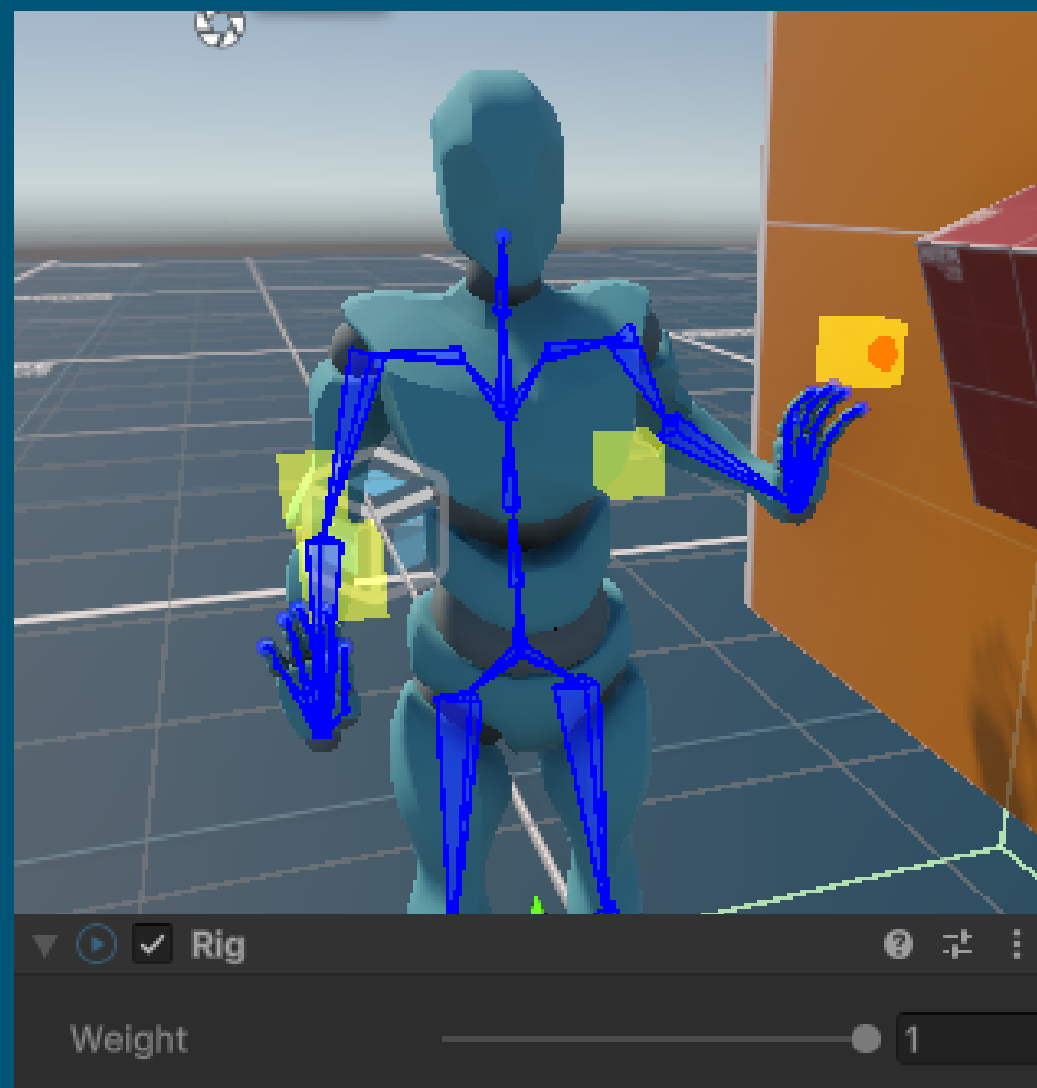
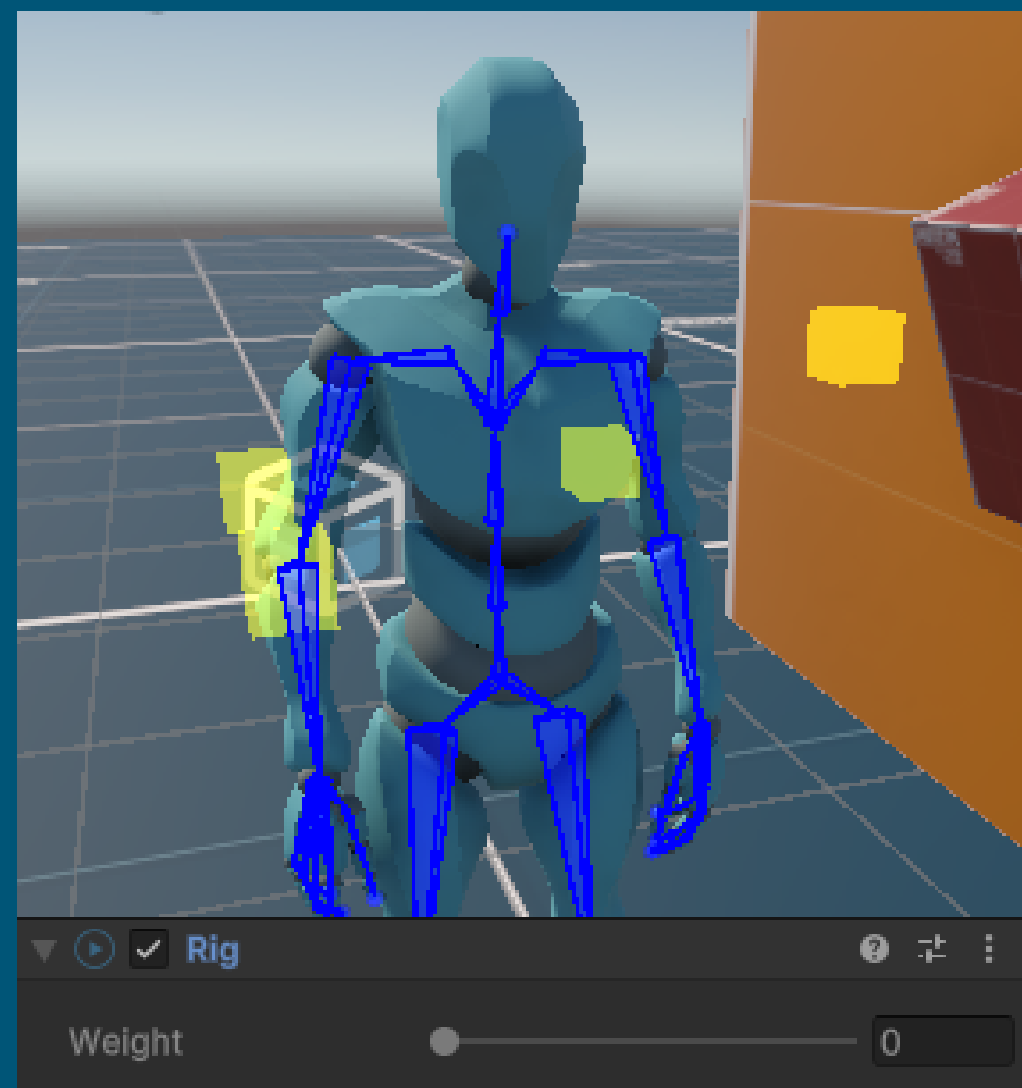
5

Utilizando métodos de detecção de triggers nativos da Unity, podemos determinar o ponto mais próximo na superfície em relação ao braço e ajustar a posição do target.

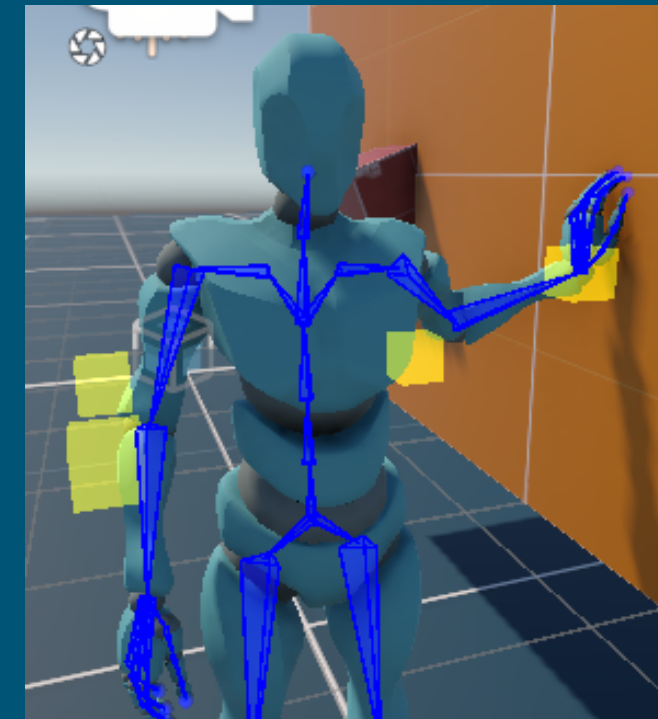


Imagens: iHeartGameDev

Podemos definir o grau de influência da animação procedural a partir do componente Rig, utilizando a propriedade "weight".



Dessa forma, podemos programaticamente definir quando a animação do braço deve começar a sobrescrever a animação original.



```
Context.CurrentIkConstraint.weight = Mathf.Lerp(Context.CurrentIkConstraint.weight, _approachWeight, _elapsedTime / _lerpDuration);
```

Obs: Existem mais passos a serem feitos em código, mas essa é a ideia.

Desenvolvimento

Agora, só falta programar o ajuste da posição do target do componente TwoBoneIK para mover a mão em direção ao objeto.

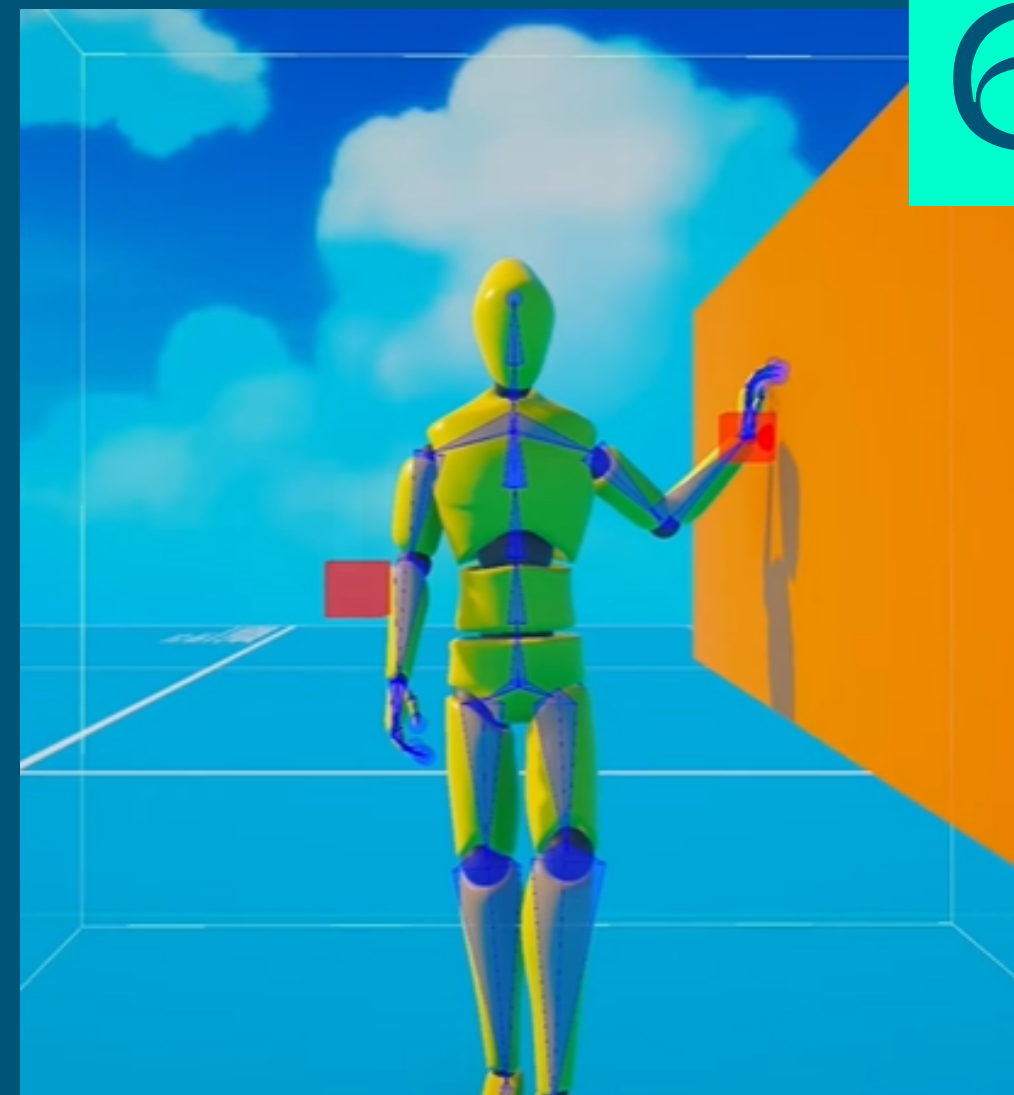
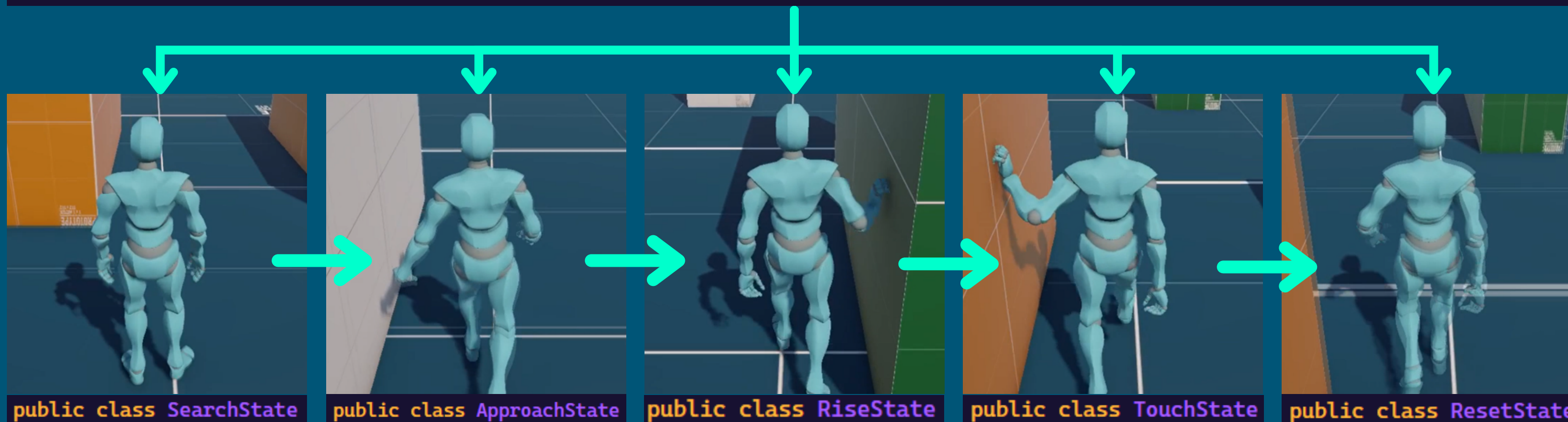


Imagem: iHeartGameDev

A animação é feita a partir de uma sequência de 5 passos (estados), criando um ciclo de animação. Isso é feito através de uma máquina de estados.

```
public class EnvironmentInteractionStateMachine
```

```
public abstract class EnvironmentInteractionState : BaseState<EnvironmentInteractionStateMachine, EEnvironmentInteractionState>
```



Obs: diagrama simplificado.

Demonstração

Considerações

- Uso de IK facilita muito a programação de animações procedurais;

O que eu aprendi de novo:

- O que é animação procedural. Como criar uma na Unity.
- FK e IK e a diferença entre ambas.
- Utilizar Animation Rigging na Unity e seus componentes.
- Programar uma máquina de estados.
- Outros pequenos detalhes sobre a Unity, como algumas propriedades, métodos da classe Vector3 e Quaternion.

Considerações

Vantagens

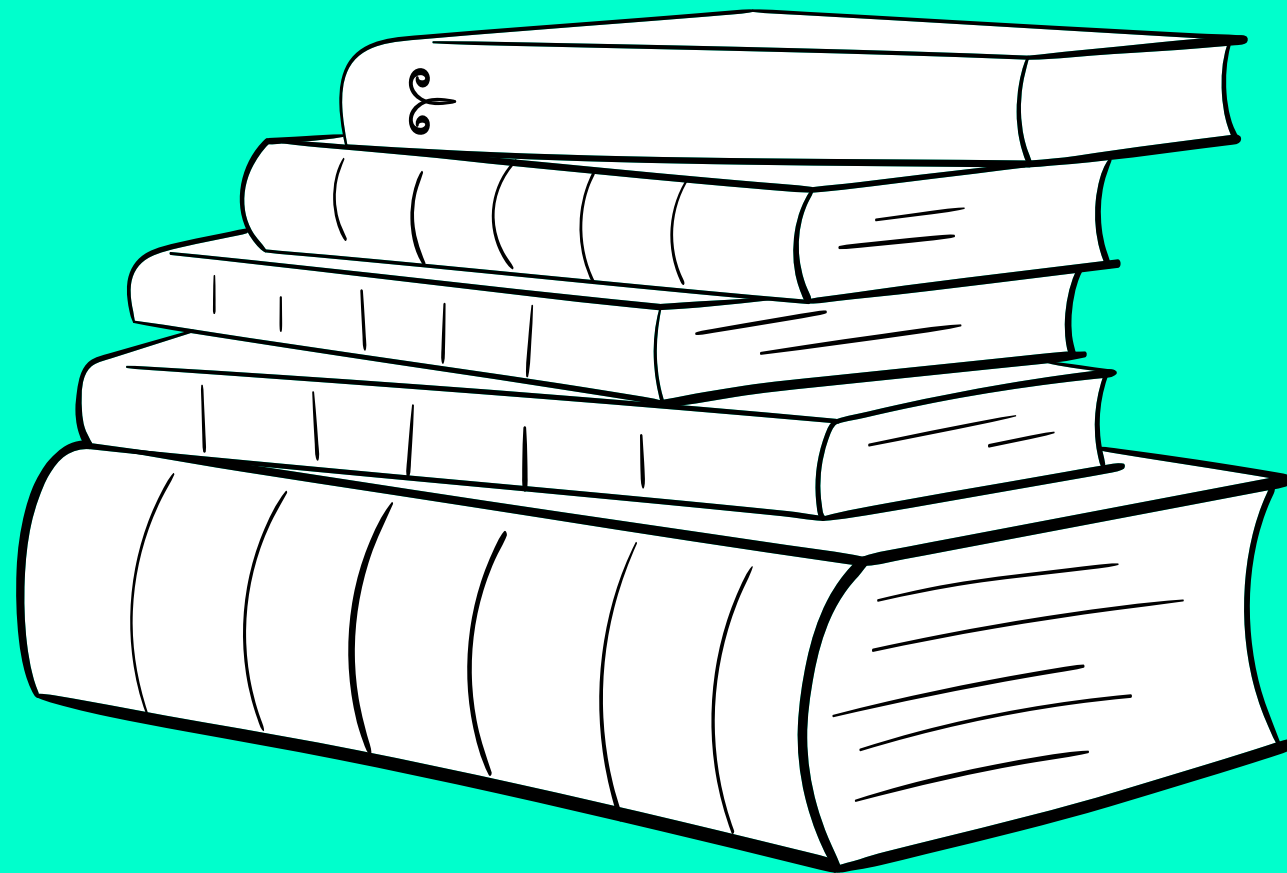
- Alta adaptabilidade ao cenário.
- Animações realistas.

Desvantagens:

- Alta complexidade de implementar em comparação a animações por keyframes.

Dificuldades:

- Código complexo (máquina de estados).



Referências

[1] - BAILEY, M. Simple Keyframe Animation. Disponível em: <<https://web.engr.oregonstate.edu/~mjb/cs491/Handouts/keyframe.1pp.pdf>>. Acesso em: 2 dez. 2025.

[2] - GREGORY, J. Game engine architecture. Boca Raton ; London ; New York: Crc Press, Taylor & Francis Group, 2019.

[3] - <https://docs.unity3d.com/Manual/InverseKinematics.html>

[4] - https://www.youtube.com/watch?v=cg_vXWPaG8A

Fim!